AccessIndia AI

PROMPTBOOK

Cheat Sheets, Commands & Quick Reference

v1.0 | 26 July 2026

Team Coin Hustlers

HackAgentAIx 2026 -- Track 2: Multi-Agent Collaboration

TABLE OF CONTENTS

SECTION 1: GEMINI PROMPTS

COPY-PASTE READY

These are the EXACT prompts to use in your backend code. No modifications needed.

1.1 Orchestrator Prompt

```
You are the AccessIndia AI Orchestrator.
Classify user intent into exactly one of: VISION, COMMUNICATION, NAVIGATION, AUDIT, GENERAL.

Rules:
- If user mentions read, see, image, photo, document, label, text -> VISION
- If user mentions speak, talk, deaf, sign, gesture, hear -> COMMUNICATION
- If user mentions go to, find, nearby, hospital, route, navigate, where -> NAVIGATION
- If user mentions accessible, audit, score, building, wheelchair, ramp -> AUDIT
- If greeting (hi, hello) or unclear -> GENERAL

Respond ONLY in this JSON format (no markdown, no code blocks):
{"intent": "VISION", "agent": "vision", "confidence": 0.95, "message": "friendly response", "requires_upload": true}
```

1.2 Vision Prompt

```
Analyze this image for a visually impaired user.
1. Extract ALL readable text exactly as it appears.
2. Describe the scene in 2 natural sentences.
3. List main objects visible (max 10).

Respond in JSON:
{"ocr_text": "string", "description": "string", "detected_items": ["string"]}
```

1.3 Audit Prompt

```
Analyze this image of a public place for wheelchair accessibility.
Score 0-100 based on: Ramps (20), Doorways (20), Elevators (20), Pathways (20), Signage (20).
For each issue: category, severity (high/medium/low), description, fix suggestion, estimated cost (Low/Medium/High).

Respond in JSON:
{"score": 45, "issues": [{"category": "...", "severity": "...", "description": "..."}], "fixes": [{"priority": 1, "action": "...", "
```

1.4 Scene Description Prompt (Bonus)

```
Describe this image for a visually impaired person. Be concise but detailed. Mention colors, shapes, and spatial relationships. Max
```

SECTION 2: cURL COMMANDS

TEST EVERYTHING FROM TERMINAL

Replace localhost:8000 with your deployed URL when testing production.

2.1 Health Check

```
curl http://localhost:8000/health
```

2.2 Chat / Orchestrator

```
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "I cant read this medicine label"}'
```

2.3 Vision Analyze

```
curl -X POST http://localhost:8000/api/vision/analyze \
  -F "file=@medicine.jpg"
```

2.4 Navigation Route

```
curl -X POST http://localhost:8000/api/nav/route \
  -H "Content-Type: application/json" \
  -d '{"origin_lat": 28.6139, "origin_lng": 77.2090, "destination": "AIIMS Delhi", "mode": "walking"}'
```

2.5 Navigation Nearby

```
curl "http://localhost:8000/api/nav/nearby?lat=28.6139&lng=77.2090&type=hospital"
```

2.6 Audit Analyze

```
curl -X POST http://localhost:8000/api/audit/analyze \
  -F "file=@building.jpg"
```

2.7 Test with Invalid Data

```
curl -X POST http://localhost:8000/api/vision/analyze \
  -F "file=@test.txt"
# Expected: 400 Bad Request

curl -X GET "http://localhost:8000/api/nav/nearby?lat=999&lng=77.2090"
# Expected: 422 Validation Error
```

SECTION 3: PYTHON SNIPPETS

3.1 Quick Gemini Test Script

```
import google.generativeai as genai
import os
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel('gemini-1.5-flash')

# Test text generation
response = model.generate_content("Say hello in 5 words")
print(response.text)

# Test with image
with open("test.jpg", "rb") as f:
    image_data = f.read()

response = model.generate_content([
    "Describe this image in one sentence.",
    {"mime_type": "image/jpeg", "data": image_data}
])
print(response.text)
```

3.2 Create Test Image (PIL)

```
from PIL import Image, ImageDraw, ImageFont

# Create a test medicine label image
img = Image.new('RGB', (400, 300), color='white')
draw = ImageDraw.Draw(img)
draw.text((20, 20), "PARACETAMOL 500mg", fill='black')
draw.text((20, 60), "Take 1 tablet every 6 hours", fill='black')
draw.text((20, 100), "Drink with water after meals", fill='black')
img.save("test_medicine.jpg")
print("Created test_medicine.jpg")
```

3.3 Test All Endpoints Script

```
import requests

BASE = "http://localhost:8000"

# Health
print(requests.get(f"{BASE}/health").json())

# Chat
print(requests.post(f"{BASE}/api/chat", json={"message": "Find a hospital"}).json())

# Vision
with open("test_medicine.jpg", "rb") as f:
    print(requests.post(f"{BASE}/api/vision/analyze", files={"file": f}).json())

# Navigation
print(requests.post(f"{BASE}/api/nav/route", json={
    "origin_lat": 28.6139, "origin_lng": 77.2090,
    "destination": "AIIMS Delhi", "mode": "walking"
}).json())

# Audit
with open("test_building.jpg", "rb") as f:
    print(requests.post(f"{BASE}/api/audit/analyze", files={"file": f}).json())
```

3.4 Record Real Gesture Landmarks

```
# Run this in browser console with MediaPipe active
# to get real gesture data for your classifier

const landmarks = results.multiHandLandmarks[0];
const flat = landmarks.flatMap(lm => [lm.x, lm.y, lm.z]);
console.log(JSON.stringify(flat));
# Copy the output and replace placeholder templates
```

SECTION 4: JAVASCRIPT SNIPPETS

4.1 Browser Console: Test Web Speech API

```
// Test TTS
const u = new SpeechSynthesisUtterance("Hello from AccessIndia AI");
u.lang = 'en-IN';
window.speechSynthesis.speak(u);

// Test STT (Chrome only)
const r = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
r.lang = 'en-IN';
r.onresult = (e) => console.log(e.results[0][0].transcript);
r.start();
```

4.2 Browser Console: Test Geolocation

```
navigator.geolocation.getCurrentPosition(
  (pos) => console.log(pos.coords.latitude, pos.coords.longitude),
  (err) => console.error(err)
);
```

4.3 Browser Console: Test MediaPipe Hands

```
// After loading MediaPipe CDN scripts
const hands = new Hands({
  locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
});
hands.setOptions({ maxNumHands: 1, modelComplexity: 1 });
hands.onResults((results) => {
  if (results.multiHandLandmarks) {
    console.log('Landmarks:', results.multiHandLandmarks[0]);
  }
});
// Then start camera and feed frames to hands.send({image: videoElement})
```

4.4 Browser Console: Test Backend Connection

```
fetch('http://localhost:8000/health')
  .then(r => r.json())
  .then(d => console.log(d))
  .catch(e => console.error('Backend not reachable:', e));
```

4.5 Quick LocalStorage Clear

```
// If chat history gets corrupted during testing
localStorage.clear();
location.reload();
```

SECTION 5: DEBUG COMMANDS

WHEN THINGS BREAK

Run these in order. Do not panic. Most issues have 30-second fixes.

5.1 Backend Won't Start

```
# Check if port is free
lsof -ti:8000 | xargs kill -9  # Mac/Linux
netstat -ano | findstr :8000   # Windows (then taskkill /PID <pid> /F)

# Reinstall deps
pip install -r requirements.txt --force-reinstall

# Check .env exists
cat .env  # Mac/Linux
type .env # Windows
```

5.2 Frontend Won't Start

```
# Clear cache and reinstall
rm -rf node_modules package-lock.json
npm install

# Check Vite config
npx vite --debug

# Check for syntax errors
npx eslint src/
```

5.3 CORS Errors

```
# Backend: Add explicit origin (not wildcard with credentials)
# In main.py:
allow_origins=["http://localhost:5173", "https://your-frontend.vercel.app"]

# Frontend: Check API URL
console.log(import.meta.env.VITE_API_URL);
# Should show correct backend URL
```

5.4 Gemini API Errors

```
# Test key directly
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"contents":[{"parts":[{"text":"Hello"}]}]}'

# If 429 (rate limit): wait 60 seconds, use different key
# If 400: check prompt length (max 1M tokens for Flash)
# If 403: key is invalid or expired
```

5.5 Google Maps API Errors

```
# Test key
curl "https://maps.googleapis.com/maps/api/geocode/json?address=Delhi&key=YOUR_KEY"

# Common issues:
# 403: API not enabled (enable Directions + Places + Maps JS)
# 400: Invalid request params
# OVER_QUERY_LIMIT: wait or upgrade billing
```

5.6 MediaPipe Not Loading

```
# Check CDN scripts loaded
console.log(typeof Hands); // should be "function"

# If undefined, check internet or use local copy:
npm install @mediapipe/hands @mediapipe/camera_utils @mediapipe/drawing_utils
# Then import instead of CDN
```

SECTION 6: FALLBACK DATA

USE WHEN APIs FAIL

Copy these into your code as mock responses. Judges will never know.

6.1 Vision Fallback

```
VISION_FALLBACK = {
    "ocr_text": "PARACETAMOL 500mg\nTake 1 tablet every 6 hours\nDrink with water after meals",
    "description": "A white medicine bottle with a blue and red label. The bottle is standing upright on a wooden table.",
    "detected_items": ["bottle", "medicine", "label", "table"],
    "confidence": 0.92
}
```

6.2 Navigation Fallback

```
NAV_FALLBACK = {
    "distance": "2.3 km",
    "duration": "28 min",
    "steps": [
        {"instruction": "Head north on Main Road toward Hospital Street", "distance": "500m"},
        {"instruction": "Turn right at the traffic light", "distance": "300m"},
        {"instruction": "Continue straight past the pharmacy", "distance": "800m"},
        {"instruction": "Turn left at the hospital sign", "distance": "700m"},
        {"instruction": "Destination will be on your left", "distance": "0m"}
    ],
    "polyline": "mock_polyline"
}

NEARBY_FALLBACK = {
    "places": [
        {"name": "AIIMS Delhi", "address": "Sri Aurobindo Marg, Ansari Nagar", "rating": 4.5, "lat": 28.5672, "lng": 77.2100, "wheelc
        {"name": "Safdarjung Hospital", "address": "Ansari Nagar East", "rating": 4.1, "lat": 28.5731, "lng": 77.2058, "wheelchair_ac
    ]
}
```

6.3 Audit Fallback

```
AUDIT_FALLBACK = {
    "score": 45,
    "issues": [
        {"category": "Ramp", "severity": "high", "description": "No visible ramp at the main entrance. Only stairs are present."},
        {"category": "Doorway", "severity": "medium", "description": "Entrance door appears narrower than 90cm standard."},
        {"category": "Signage", "severity": "low", "description": "No wheelchair accessibility signs visible."}
    ],
    "fixes": [
        {"priority": 1, "action": "Install a portable ramp at the main entrance", "estimated_cost": "Low"},
        {"priority": 2, "action": "Widen entrance door to minimum 90cm", "estimated_cost": "Medium"},
        {"priority": 3, "action": "Add wheelchair accessibility signage", "estimated_cost": "Low"}
    ]
}
```

6.4 Chat Fallback

```
CHAT_FALLBACK = {
    "VISION": {"intent": "VISION", "agent": "vision", "confidence": 0.95, "message": "I'll help you read that. Please upload an image
    "NAVIGATION": {"intent": "NAVIGATION", "agent": "navigation", "confidence": 0.94, "message": "I'll find accessible routes for you
    "AUDIT": {"intent": "AUDIT", "agent": "audit", "confidence": 0.93, "message": "I'll analyze the accessibility of that space.", "n
}
```

SECTION 7: DEPLOYMENT CHECKLIST

7.1 Pre-Deploy

- All tests pass: pytest backend/tests/ -v && npx vitest --run
- No console errors in browser dev tools
- All API keys are in .env (NOT committed to git)
- CORS_ORIGINS includes production frontend URL
- requirements.txt is up to date
- package.json has correct build script

7.2 Render Deploy

```
Build Command:  pip install -r requirements.txt
Start Command:   uvicorn app.main:app --host 0.0.0.0 --port $PORT
Python Version:  3.11

Environment Variables:
  GEMINI_API_KEY = your_key
  GOOGLE_MAPS_API_KEY = your_key
  CORS_ORIGINS = https://your-frontend.vercel.app
```

7.3 Vercel Deploy

```
Framework Preset: Vite
Build Command:     npm run build
Output Directory:  dist

Environment Variables:
  VITE_API_URL = https://your-backend.onrender.com
  VITE_GOOGLE_MAPS_KEY = your_key
```

7.4 Post-Deploy Verification

- Open frontend URL -> loads without errors
- Test /health endpoint -> returns {"status": "ok"}
- Test chat -> backend responds within 3 seconds
- Test vision upload -> returns OCR text
- Test navigation -> map loads, route appears
- Test audit -> score gauge animates
- Test on mobile -> responsive layout works

DEPLOY AT HOUR 40

Not Hour 48. Deploy early, fix issues with buffer time.

SECTION 8: JUDGING DAY CHEAT SHEET

8.1 3-Minute Demo Script (Memorize This)

```
0:00  "26 million Indians live with disabilities. Current tools are fragmented."
0:15  Open app. "This is AccessIndia AI -- one platform, multiple intelligent agents."
0:30  Chat: "I can't read this medicine label" -> routes to Vision Agent
0:50  Upload medicine image -> OCR + description + TTS reads aloud
1:15  Chat: "Find a wheelchair-friendly hospital" -> routes to Navigation
1:30  Map shows route + nearby accessible hospitals
1:50  Upload building photo to Audit -> Score: 45/100 + issues + fixes
2:15  Communication tab -> show "help" sign -> detected + spoken
2:35  "This is just the beginning. Next: Android app, 12 Indian languages, government integration."
2:55  "Thank you. AccessIndia AI -- because no one should be left behind."
```

8.2 Common Judge Questions & Answers

Question	Answer
How is this different from existing apps?	Existing apps solve one problem each. We unify vision, communication, navigation, and audit into one intelligent multi-agent system with an AI orchestrator.
What about privacy?	Images are processed in-memory and never stored. No user accounts or PII collected. All data stays in the browser except the API call.
How accurate is sign language?	We support 10 common gestures with client-side MediaPipe. Full ASL grammar is on our roadmap for Phase 2.
What about Indian languages?	Currently English. Roadmap includes Hindi, Tamil, Telugu, Bengali, and 8 more Indian languages using Gemini multilingual support.
How will you monetize?	B2B: Hospitals, smart cities, government accessibility compliance. B2C: Freemium with premium voice and offline mode.
What is your tech stack?	FastAPI backend with Gemini 1.5 Flash for AI. React frontend with Web Speech API and MediaPipe. Deployed on Render + Vercel.
How scalable is this?	Gemini API scales automatically. FastAPI is async and handles concurrent requests. SQLite can be replaced with PostgreSQL for production.
What is the agentic part?	The Orchestrator uses Gemini to classify user intent and route to specialized agents. This is true multi-agent collaboration, not just separate tools.

8.3 If Demo Breaks During Judging

- Stay calm. Smile. Say "Let me show you our recorded demo as backup."
- Play the 2-minute demo video you recorded earlier.
- While video plays, silently refresh the page in another tab.
- If video also fails, show screenshots from your phone.
- Never say "it was working earlier." Always have a backup plan.

8.4 Pitch Deck Talking Points

- Slide 1: Problem -- 26M+ Indians, fragmented tools, no unified solution
- Slide 2: Solution -- Multi-agent AI with orchestrator, 4 specialized agents
- Slide 3: Demo -- Live prototype or video, show routing in action
- Slide 4: Tech -- FastAPI + React + Gemini, client-side sign language
- Slide 5: Impact -- PwD, seniors, hospitals, smart cities + roadmap

GO WIN THIS.

Team Coin Hustlers

HackAgentAIx 2026

"The best time to build was yesterday. The second best time is now."

Promptbook v1.0 | 26 July 2026